

STEP 1 — Preprocess ☐lowercase ☐punctuation removal ☐stopword removal ☐tokenization ☐lemmatization Explain each step.

```
import re
import string
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer

# Download required NLTK packages
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')

# Preprocessing function
def preprocess(text, lemmatize=False):
    # Step 1: Lowercasing
    text = text.lower()

    # Step 2: Punctuation removal
    text = re.sub(f"[{string.punctuation}]", " ", text)

    # Step 3: Tokenization
    tokens = word_tokenize(text)

    # Step 4: Stopword Removal
    stop_words = set(stopwords.words('english'))
    tokens = [word for word in tokens if word not in stop_words]

    # Step 5: Lemmatization (optional)
    if lemmatize:
        lemmatizer = WordNetLemmatizer()
        tokens = [lemmatizer.lemmatize(word) for word in tokens]

    return tokens
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
[nltk_data] Downloading package wordnet to /root/nltk_data...
```

STEP 2 — Feature Representation Students must use: ☐TF-IDF for cosine similarity ☐Bag-of-Words for Jaccard similarity Explain WHY both are needed.

```
from sklearn.feature_extraction.text import TfidfVectorizer, CountVectorizer

# Example text data
documents = ["The cat is on the mat.", "The dog is on the mat."]

# Step 1: TF-IDF Vectorization
tfidf_vectorizer = TfidfVectorizer()
tfidf_matrix = tfidf_vectorizer.fit_transform(documents)

# Step 2: Bag-of-Words Vectorization
bow_vectorizer = CountVectorizer()
bow_matrix = bow_vectorizer.fit_transform(documents)

# Print the TF-IDF and BoW matrices
print("TF-IDF Matrix:\n", tfidf_matrix.toarray())
```

```
print("BoW Matrix:\n", bow_matrix.toarray())
```

TF-IDF Matrix:

```
[[0.46913173 0.33379109 0.33379109 0.33379109 0.66758217]
 [0.46913173 0.33379109 0.33379109 0.33379109 0.66758217]]
```

BoW Matrix:

```
[[1 0 1 1 1 2]
 [0 1 1 1 1 2]]
```

STEP 3 — Cosine Similarity □compare each document pair □highlight top 5 most similar document pairs
□interpret scores

```
from sklearn.metrics.pairwise import cosine_similarity

# Step 1: Compute Cosine Similarity
cos_sim = cosine_similarity(tfidf_matrix)

# Step 2: Find Top 5 Most Similar Document Pairs
import numpy as np

# Get pairs of document indices with highest similarity
similarity_scores = cos_sim[np.triu_indices(len(cos_sim), 1)] # upper triangular matrix
sorted_indices = np.argsort(similarity_scores)[-5:][::-1]

# Print top 5 similar document pairs
for index in sorted_indices:
    i, j = np.triu_indices(len(cos_sim), 1)[0][index], np.triu_indices(len(cos_sim), 1)[1][index]
    print(f"Documents {i+1} and {j+1}: Similarity = {similarity_scores[index]}")
```

Documents 1 and 2: Similarity = 0.7799154245579976

STEP 4 — Jaccard Similarity □calculate Jaccard for same document pairs □observe how overlapping word sets affect scores

```
# Jaccard Similarity Function
def jaccard_similarity(set1, set2):
    intersection = len(set1.intersection(set2))
    union = len(set1.union(set2))
    return intersection / union

# Example sets of words (documents)
doc1_words = set(["the", "cat", "is", "on", "mat"])
doc2_words = set(["the", "dog", "is", "on", "mat"])

# Calculate Jaccard Similarity
jaccard_sim = jaccard_similarity(doc1_words, doc2_words)
print("Jaccard Similarity:", jaccard_sim)
```

Jaccard Similarity: 0.6666666666666666

STEP 5 — WordNet Semantic Similarity □identify synonyms/paraphrased sentences □show how semantic similarity detects meaning similarity (Apply to at least 10 sentence pairs)

```
from nltk.corpus import wordnet as wn

# WordNet Similarity Function
def wordnet_similarity(word1, word2):
    synsets1 = wn.synsets(word1)
    synsets2 = wn.synsets(word2)
```

```
max_similarity = 0
for syn1 in synsets1:
    for syn2 in synsets2:
        similarity = wn.wup_similarity(syn1, syn2)
        if similarity:
            max_similarity = max(max_similarity, similarity)
    return max_similarity

# Example usage
word1 = "cat"
word2 = "kitten"
similarity = wordnet_similarity(word1, word2)
print(f"WordNet Similarity between '{word1}' and '{word2}':", similarity)
```

```
WordNet Similarity between 'cat' and 'kitten': 0.6
```